

LEARNIFY

Memoria Técnica

Proyecto Integrador Profesional — DAW 2

Ángel Ríos · DIGITECH FP · 2024

Repositorio: github.com/Angelrp2/learnify

1. Introducción y Objetivos

Learnify es una plataforma educativa para aprender programación y desarrollo web. El objetivo principal es crear un sistema moderno que permita a los estudiantes explorar cursos, filtrar por categoría y dificultad, y acceder a contenido educativo de calidad.

El proyecto demuestra la aplicación de conceptos avanzados de JavaScript, arquitectura de software desacoplada, y buenas prácticas de desarrollo web profesional.

2. Planificación y Metodología

Arquitectura Headless CMS

Se eligió una arquitectura desacoplada donde el backend (Strapi) gestiona los datos y el frontend (Astro) renderiza el contenido. Esta separación proporciona flexibilidad, escalabilidad y mejor rendimiento.

- El frontend se genera estáticamente en build time, mejorando el SEO y el rendimiento.
- El backend puede modificarse sin afectar el frontend.
- La base de datos está protegida detrás de una API REST.
- Los cambios en el contenido se reflejan automáticamente al reconstruir el sitio.

Tecnologías Seleccionadas

Frontend: Astro (generación estática SSG con JavaScript mínimo en islands) | **Backend:** Strapi (CMS Headless con API REST automática) | **Estilos:** Tailwind CSS integrado con variables CSS globales | **Base de datos:** SQLite en desarrollo, PostgreSQL en producción | **Control de versiones:** Git con GitHub

Fases de Desarrollo

- Fase 1: Diseño de la arquitectura y estructura de datos en Strapi
- Fase 2: Creación de colecciones y relaciones en Strapi
- Fase 3: Desarrollo del frontend con Astro y sistema de estilos
- Fase 4: Implementación de funcionalidades (búsqueda, filtros, paginación)
- Fase 5: Optimización SEO, sitemap dinámico y robots.txt
- Fase 6: Documentación, revisión de código y despliegue

3. Diseño y Tecnologías Empleadas

Estructura de Base de Datos

La base de datos relacional contiene seis colecciones principales con relaciones 1:N entre ellas:

Colección	Descripción	Relaciones
Categories	Categorías de cursos	1:N con Courses y BlogPosts
Instructors	Perfiles de profesores	1:N con Courses y BlogPosts
Courses	Cursos principales	N:1 con Categories e Instructors
Lessons	Lecciones de cada curso	N:1 con Courses
BlogPosts	Artículos educativos	N:1 con Categories e Instructors
ContactSubmissions	Mensajes del formulario	Independiente

Conceptos JavaScript Aplicados

Desestructuración

Se utiliza para extraer propiedades de los objetos de la API de Strapi de forma concisa, mejorando la legibilidad del código en todos los componentes.

```
const { attributes } = course;
const { title, description, slug, difficulty } = attributes;
```

Métodos de Array

El proyecto implementa map para transformar datos, filter para búsquedas y filtros combinados, sort para ordenamiento y slice para paginación.

```
const beginnerCourses = courses.filter(c => c.attributes.difficulty === 'beginner');
const sorted = courses.sort((a, b) => a.attributes.title.localeCompare(b.attributes.title));
const page = courses.slice(start, end);
```

Async/Await con Try/Catch

Todas las llamadas a la API de Strapi utilizan async/await. El manejo de errores con try/catch está implementado en cada operación asíncrona, previniendo que la aplicación se bloquee ante fallos de red.

```
export async function fetchCourses() {
  try {
    const response = await fetch(`${STRAPI_URL}/api/courses?populate=*`);
    if (!response.ok) throw new Error(`Error: ${response.status}`);
    const data = await response.json();
    return data.data || [];
  } catch (error) {
    console.error('Error fetching courses:', error);
  }
}
```

```
    return [];  
  }  
}
```

Expresiones Regulares

Se utilizan en el buscador dinámico y en la validación del formulario de contacto. La búsqueda es case-insensitive y opera sobre múltiples campos simultáneamente.

```
function searchCourses(courses, query) {  
  const regex = new RegExp(query, 'i');  
  return courses.filter(course =>  
    regex.test(course.attributes.title) ||  
    regex.test(course.attributes.description)  
  );  
}  
  
function validateEmail(email) {  
  const regex = /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/;  
  return regex.test(email);  
}
```

Normalización de Texto

Se implementa la función slugify para generar URLs amigables automáticamente desde los títulos, eliminando acentos, caracteres especiales y espacios.

```
function slugify(text) {  
  return text  
    .toLowerCase()  
    .normalize('NFD')  
    .replace(/[\u0300-\u036f]/g, '')  
    .replace(/[^\\w\\s-]/g, '')  
    .replace(/\\s+/g, '-')  
    .trim();  
}
```

Paginación

```
function paginateCourses(courses, page = 1, pageSize = 6) {  
  const start = (page - 1) * pageSize;  
  return {  
    courses: courses.slice(start, start + pageSize),  
    totalPages: Math.ceil(courses.length / pageSize),  
    hasNextPage: page < Math.ceil(courses.length / pageSize),  
  };  
}
```

4. Sistema de Estilos

Arquitectura CSS — Variables :root + Tailwind CSS

El sistema de estilos combina dos capas complementarias: variables CSS globales en :root para los valores del tema, e integración de Tailwind CSS para las clases utilitarias. Esta combinación permite cambiar el tema completo modificando únicamente las variables, sin tocar los componentes individuales.

Variables Globales en :root (global.css)

Las variables definen todos los valores reutilizables del sistema: colores, espaciado y tipografía. El dark mode se implementa sobrescribiendo estas variables con la clase html.dark.

```
:root {
  --color-primary: #2563eb;
  --color-secondary: #64748b;
  --color-success: #10b981;
  --color-error: #ef4444;
  --bg-primary: #ffffff;
  --bg-secondary: #f8fafc;
  --text-primary: #1e293b;
  --text-secondary: #64748b;
  --border: #e2e8f0;
  --spacing-xs: 0.25rem;
  --spacing-md: 1rem;
  --spacing-section: 4rem;
  --font-sans: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
}

html.dark {
  --bg-primary: #0f172a;
  --bg-secondary: #1e293b;
  --text-primary: #f1f5f9;
  --text-secondary: #cbd5e1;
  --border: #334155;
}
```

Integración con Tailwind CSS (tailwind.config.mjs)

Tailwind extiende su configuración para referenciar las variables CSS. Esto permite usar clases como bg-primary o text-muted directamente en los componentes, manteniendo la coherencia con el tema definido en :root.

```
export default {
  content: ['./src/**/*.astro', './src/**/*.html', './src/**/*.js', './src/**/*.ts'],
  darkMode: 'class',
  theme: {
    extend: {
      colors: {
        primary: 'var(--color-primary)',
```

```
secondary: 'var(--color-secondary)',
'bg-base': 'var(--bg-primary)',
'bg-muted': 'var(--bg-secondary)',
'text-base': 'var(--text-primary)',
'text-muted': 'var(--text-secondary)',
border: 'var(--border)',
},
},
},
};
```

Componentes Reutilizables

Se crearon cinco componentes Astro reutilizables que usan clases Tailwind directamente, sin bloques de CSS interno:

- **Button.astro** — variantes primary, secondary y outline; tamaños sm, md y lg; estado disabled
- **Card.astro** — imagen, título, descripción, badge de dificultad y enlace
- **Section.astro** — wrapper semántico con props class e id
- **Container.astro** — centra el contenido con max-width configurable (sm/md/lg/xl)
- **BaseLayout.astro** — layout base con navegación, footer y toggle dark/light mode

Dark Mode y Responsive

El dark mode se activa añadiendo la clase dark al elemento html mediante JavaScript. La preferencia se guarda en localStorage para que persista entre visitas. El diseño es mobile-first con breakpoints en 768px (md) y 1024px (lg) usando las clases responsivas de Tailwind.

5. Estrategia SEO Aplicada

SEO On-Page

Cada página del sitio cumple los requisitos de SEO on-page. Los meta tags se generan dinámicamente desde los datos de Strapi para cursos y artículos, garantizando que cada página tenga metadatos únicos y relevantes.

- H1 único y descriptivo en cada página, que contiene la keyword principal
- Jerarquía correcta de encabezados: H1 → H2 → H3
- Meta title optimizado (máximo 60 caracteres) generado desde Strapi
- Meta description estratégica (máximo 160 caracteres) generada desde Strapi
- URLs amigables con slugs: /cursos/introduccion-a-javascript
- Atributos ALT descriptivos en todas las imágenes
- Enlazado interno coherente entre cursos, categorías y blog

- Open Graph tags para mejorar la apariencia al compartir en redes sociales

SEO Técnico

- sitemap.xml generado dinámicamente en build time consultando Strapi, incluyendo todas las URLs de cursos y blog
- robots.txt que bloquea /admin y /api y apunta al sitemap
- HTML semántico: header, main, footer, article, section, nav
- URL canónica en cada página para evitar contenido duplicado
- Optimización de imágenes con lazy loading
- Arquitectura limpia sin duplicidades innecesarias

Contenido

El contenido está redactado de forma natural, sin patrones de IA evidentes. Las descripciones de cursos y artículos de blog tienen voz propia, las meta descriptions son persuasivas y orientadas al usuario, y los títulos responden a intención de búsqueda real.

6. Funcionalidades Implementadas

Búsqueda Dinámica con Regex

El buscador carga todos los cursos en el cliente en el momento de la página y filtra en tiempo real usando expresiones regulares. La búsqueda es case-insensitive y opera sobre título y descripción simultáneamente.

Filtros y Ordenación

Los usuarios pueden filtrar por dificultad (principiante, intermedio, avanzado) y por categoría. Los filtros se combinan lógicamente con AND. La ordenación permite organizar por título, fecha o dificultad.

Paginación

El listado de cursos está paginado con 6 elementos por página. La navegación muestra botones anterior, siguiente y números de página. Al cambiar de página hace scroll al inicio automáticamente.

Validación de Formulario

El formulario de contacto valida todos los campos con expresiones regulares y muestra feedback visual inmediato. Se valida email con regex, longitud mínima en nombre, asunto y mensaje, y se requiere consentimiento RGPD explícito.

Tiempo de Lectura

Los artículos del blog calculan automáticamente el tiempo de lectura estimado basándose en el conteo de palabras a 200 palabras por minuto.

API Externa — GitHub

La página de recursos consume la GitHub API en tiempo real para mostrar repositorios educativos etiquetados con javascript-tutorial, con manejo de errores mediante try/catch.

7. Seguridad y Cumplimiento RGPD

Strapi utiliza ORM que previene inyecciones SQL automáticamente. Las variables de entorno protegen los secretos (JWT, API keys). Se validan todas las entradas de usuario en cliente con regex. El formulario de contacto requiere consentimiento informado explícito conforme al RGPD.

8. Conclusión

Learnify demuestra la aplicación profesional de conceptos avanzados de JavaScript, arquitectura moderna de software desacoplada, y buenas prácticas de desarrollo web. El proyecto cumple todos los requisitos mínimos del proyecto integrador y proporciona una base sólida para futuras extensiones.

La arquitectura desacoplada permite escalar el frontend y el backend de forma independiente. El sistema de estilos basado en variables CSS integradas con Tailwind permite adaptar el diseño a cualquier cliente modificando únicamente las variables del tema. El código es reutilizable, mantenible y está documentado en el repositorio.

9. Instrucciones de Instalación

Requisitos

Node.js 18 o superior, pnpm, Git.

Instalación Local

```
git clone https://github.com/Angelrp2/learnify.git
cd learnify

# Terminal 1 – Backend
cd strapi && pnpm install && pnpm run dev
# Accede a http://localhost:1337/admin

# Terminal 2 – Frontend
cd astro && pnpm install && pnpm run dev
# Accede a http://localhost:3000
```

Repositorio y Documentación

Repositorio GitHub: github.com/Angelrp2/learnify

- docs/ARQUITECTURA.md — Decisiones técnicas
- docs/CONTENIDOS_TECNICOS.md — Ejemplos de código
- docs/ESTILOS.md — Sistema de diseño
- docs/SEO.md — Estrategia SEO
- docs/DIAGRAMA_ER.md — Estructura de base de datos
- docs/MANUAL_INSTALACION.md — Pasos detallados de instalación